NODE.JS + EXPRESS

EJS Templating Tutorial

Generating dynamic HTML on the server with Embedded JavaScript

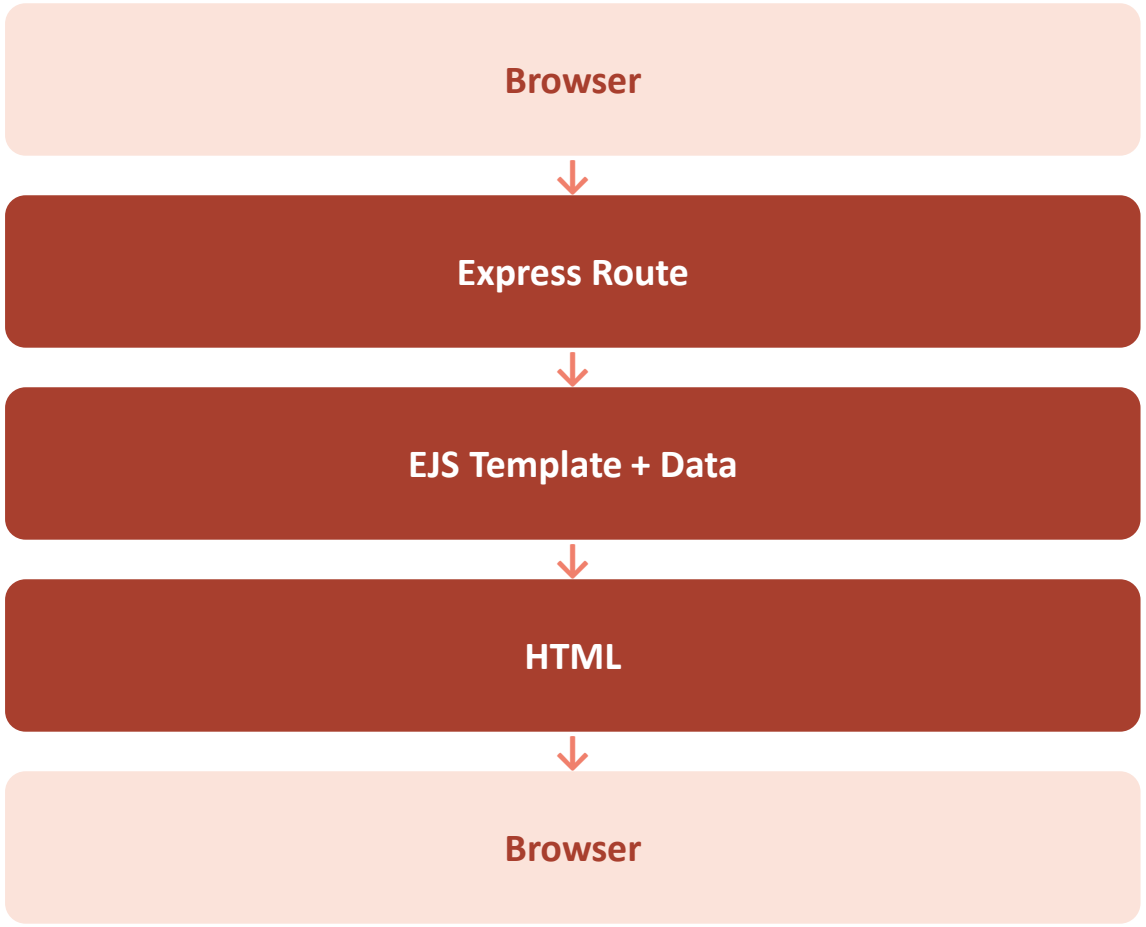
Sudhakar · Full Stack Training

CONCEPT

What Is EJS?

EJS (Embedded JavaScript Templates) lets you write plain JavaScript inside HTML to generate dynamic pages on the server, before they ever reach the browser.

Express sends data → EJS uses that data to generate HTML → the browser receives normal HTML.



1. Create the Project

1 Initialize & install

```
mkdir ejs-demo
cd ejs-demo
npm init -y
npm install express ejs
```

2 Folder structure

```
ejs-demo/
|-- app.js
`-- views/
    `-- index.ejs
```

Why this structure?

- app.js boots Express and defines your routes.
- views/ holds every .ejs template Express will render.
- Express looks in views/ by default — no extra config needed.
- The .ejs extension tells Express which rendering engine to use for that file.

2. Your First EJS Page

app.js

```
import express from "express";
const app = express();

app.set("view engine", "ejs");

app.get("/", (req, res) => {
  res.render("index", { name: "Sudhakar" });
});

app.listen(3000);
```

views/index.ejs

```
<!DOCTYPE html>
<html>
<body>
  <h1>Hello <%= name %></h1>
</body>
</html>
```

Visit http://localhost:3000 → **Hello Sudhakar**

<%= %> prints the JavaScript value straight into the HTML.

SYNTAX

Three EJS Tags To Know

<%= %>

Display value

Prints the (escaped) result of the expression into the page.

```
<h1><%= name %></h1>
```

<% %>

Execute JavaScript

Runs JS logic — loops, ifs, variables — without printing anything.

```
<% const x = 10; %>
```

<%- %>

Render raw HTML

Outputs unescaped HTML. Use carefully — it will render markup as-is.

```
<%- html %>
```

DATA

Passing Data Into a Template

Multiple values

```
res.render("index", {
  name: "Sudhakar",
  age: 30,
  city: "Vijayawada"
});
```

```
<h1><%= name %></h1>
<p>Age: <%= age %></p>
<p>City: <%= city %></p>
```

Or bundle it as one object

```
const user = {
  name: "Sudhakar",
  age: 30,
  city: "Vijayawada"
};

res.render("index", { user });
```

```
<h1><%= user.name %></h1>
<p><%= user.age %></p>
<p><%= user.city %></p>
```

LOGIC

Conditionals

Express

```
app.get("/", (req, res) => {
  res.render("index", {
    loggedIn: true
  });
});
```

views/index.ejs

```
<% if (loggedIn) { %>
  <h1>Welcome User</h1>
<% } else { %>
  <h1>Please Login</h1>
<% } %>
```

<% if (...) { %> and <% } %> are plain JavaScript blocks — everything between them is normal control flow, not a special EJS feature.

ITERATION

Looping Over an Array

Express sends an array

```
const books = [
  { id: 1, title: "Java", author: "James" },
  { id: 2, title: "Spring Boot", author: "John" },
  { id: 3, title: "MongoDB", author: "Mike" }
];

app.get("/books", (req, res) => {
  res.render("books", { books });
});
```

books.ejs

```
<h1>Books</h1>

<% books.forEach(book => { %>
  <h2><%= book.title %></h2>
  <p>Author: <%= book.author %></p>
<% }) %>
```

Output

```
Java — Author: James
Spring Boot — Author: John
MongoDB — Author: Mike
```

ITERATION

Rendering an Array as a Table

The pattern behind almost every CRUD list page.

```
<table>
  <tr><th>ID</th><th>Title</th><th>Author</th><th>Price</th></tr>

  <% books.forEach(book => { %>
    <tr>
      <td><%= book.id %></td>
      <td><%= book.title %></td>
      <td><%= book.author %></td>
      <td><%= book.price %></td>
    </tr>
  <% }) %>
</table>
```

ID	Title	Price
1	Java	499
2	Spring Boot	599
3	MongoDB	549

USER INPUT

Forms → req.body

add.ejs

```
<form action="/books" method="POST">
  <label>Title</label>
  <input type="text" name="title">
  <label>Author</label>
  <input type="text" name="author">
  <button type="submit">Add Book</button>
</form>
```

app.js

```
app.use(express.urlencoded(
  { extended: true }
));

app.post("/books", (req, res) => {
  console.log(req.body);
});
```

Submitting Title: **Java** · Author: **James**

`produces  req.body = { title: "Java", author: "James" }`

ROUTING

Dynamic URLs & Route Params

```
app.get("/books/:id", (req, res) => {
  console.log(req.params.id);
});
```

```
GET /books/10
req.params.id  →  "10"
```

In the EJS template

```
<a href="/books/<%= book.id %>">
  View
</a>
```

How it resolves

1. **book.id is 5**

   the template writes <a href="/books/5">

2. **User clicks the link**

   browser requests GET /books/5

3. **Express matches :id**

   req.params.id becomes the string "5"

Edit Form — Pre-filled Fields

app.js

```
app.get(
  "/books/edit/:id",
  (req, res) => {
    const book = books.find(
      b => b.id == req.params.id
    );
    res.render("edit", { book });
  }
);
```

edit.ejs

```
<form action="/books/edit/<%= book.id %>"
      method="POST">

  <input type="text" name="title"
         value="<%= book.title %>">

  <input type="text" name="author"
         value="<%= book.author %>">

  <input type="number" name="price"
         value="<%= book.price %>">

  <button>Update</button>
</form>
```

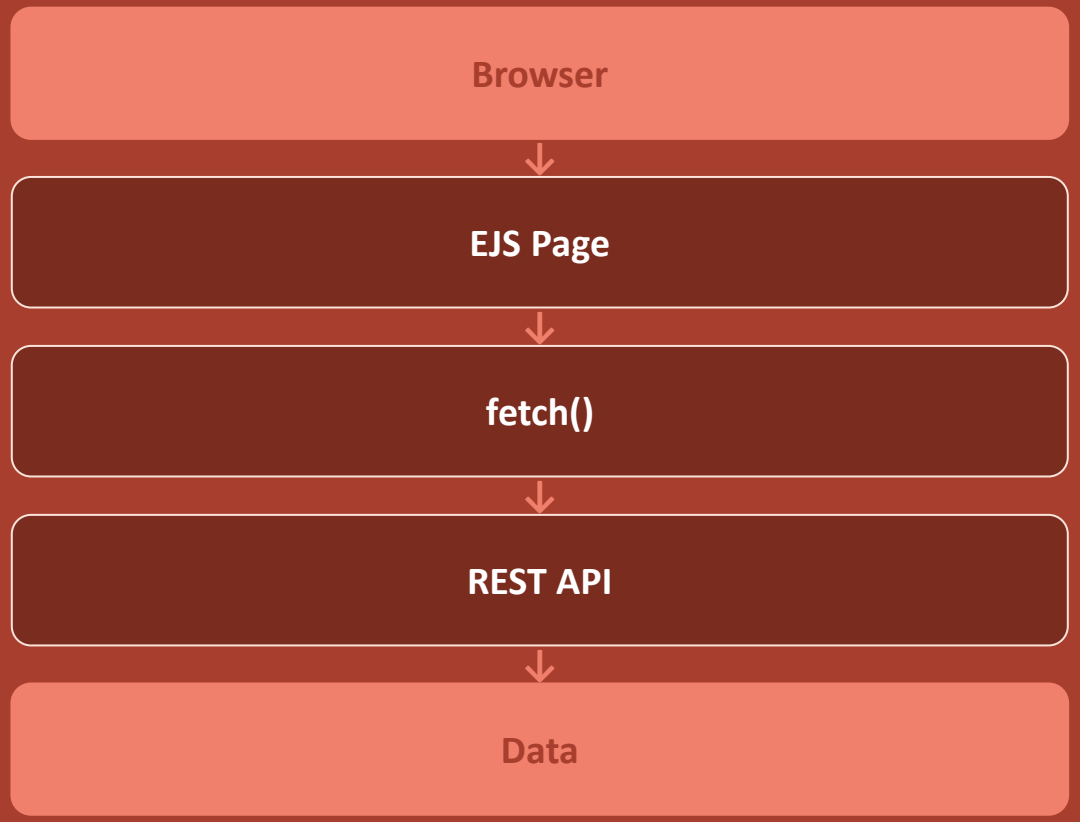
ARCHITECTURE

EJS Alongside a REST API

EJS can serve the UI while a separate REST API handles data — keeping views and API routes cleanly apart.

```
routes/
  bookApi.js

views/
  index.ejs
  add.ejs
  edit.ejs
```



PUTTING IT TOGETHER

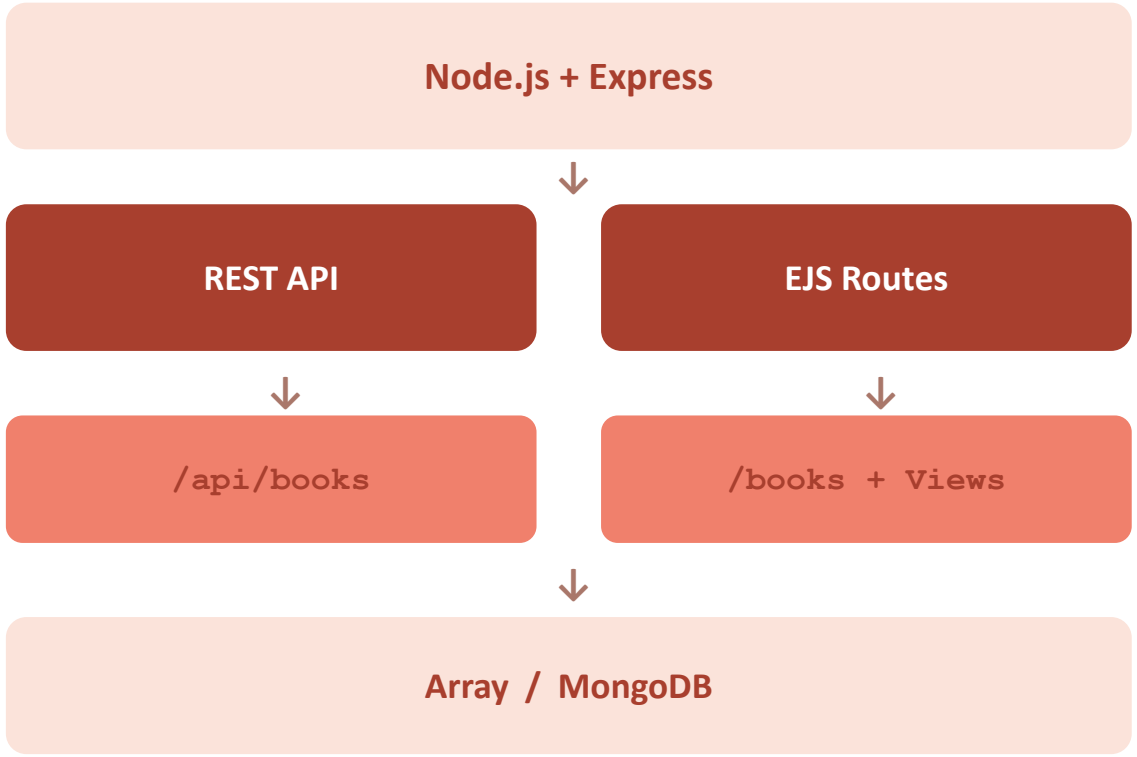
Complete EJS CRUD Concept

REST API

```
GET    /api/books
GET    /api/books/:id
POST   /api/books
PUT    /api/books/:id
DELETE /api/books/:id
```

EJS Pages

```
GET  /books
GET  /books/add
GET  /books/edit/:id
```



REFERENCE

EJS Cheat Sheet

`<%= value %>`	Print escaped value
`<%- value %>`	Print raw HTML
`<% code %>`	Execute JavaScript
`<% if (...) { %>`	Conditional
`<% } %>`	Close block
`<% array.forEach(...) %>`	Loop
`res.render()`	Render an EJS page

</>

KEY TAKEAWAY

Express sends data → EJS generates HTML → the browser receives a normal page.

Next up: connect EJS views to a real REST API and a database (MongoDB) to complete the CRUD flow.